

Question 1

Which of the following tasks is not supported by Git folders, and must be performed in your Git provider ?

- A. Clone, push to, or pull from a remote Git repository.
- B. Visually compare differences upon commit.
- C. Delete branches
- D. Create and checkout branches for development work.

Correct answer: C

C. Delete branches

Explanation

The following tasks are not supported by Databricks Repos, and must be performed in your Git provider:

- Create a pull request
- Delete branches

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 2

A data engineer got a `java.lang.OutOfMemoryError` after running `df.toPandas()` on a large Spark `DataFrame`?

Which action should the data engineer take to fix this issue?

A. Enable Adaptive Query Execution (AQE)

B. increase the number of shuffle partitions

C. Increase executor memory

D. increase the driver memory

Correct answer: D

D. increase the driver memory

Explanation

When you call `df.toPandas()` (or `df.collect()`) on a Spark `DataFrame`, Spark collects **all** the distributed data from the executor nodes across the cluster and sends it to a single node: the **driver**.

Once all that data arrives at the driver, Spark attempts to load it entirely into the driver's local JVM memory to convert it into a local Python pandas `DataFrame`. If the size of the dataset exceeds the available memory allocated to the driver, the JVM runs out of space, throwing a `java.lang.OutOfMemoryError` (OOM). So, the correct action to fix this issue is to increase the driver memory.

Why the other options won't fix it

- **Increase executor memory:** Executors handle the distributed processing. While this helps if you get an OOM during transformations (like a large join or `groupBy`), it won't help here because the failure happens after the data leaves the executors and hits the driver.

- **Increase the number of shuffle partitions:** This optimizes how data is distributed across the network during a shuffle, preventing executors from being overwhelmed. It has no effect on the final `.toPandas()` collection.

- **Enable Adaptive Query Execution (AQE):** AQE optimizes query plans dynamically at runtime (e.g., converting joins or coalescing shuffle partitions). It does not alter the fact that `.toPandas()` forces a massive, non-distributed data collection on a single node.

Question 3

A data engineer has a DataFrame, `df`, with a location column formatted as “City ZipCode”, such as “Paris 75015”. They want to create two new columns, **city** and **zip_code**, by extracting both values, then removing the original location column.

Which PySpark code snippet should the data engineer use?

A.

```
from pyspark.sql.functions import split

df = (
    df.select(split('location', ' ').alias('places'))
    .select('places[0]', 'places[1]')
)
```

B.

```
from pyspark.sql.functions import split, col

df = (
    df.withColumn("city", split(col("location"), " ")[0])
    .withColumn("zip_code", split(col("location"), " ")[1])
    .drop("location")
)
```

C.

```
from pyspark.sql.functions import explode

df = (
    df.select(explode('location').alias('places'))
    .select('places[0]', 'places[1]')
    .drop("location")
)
```

D.

```
from pyspark.sql.functions import getItem, col

df = (
    df.withColumn("city", getItem(col("location"), 0))
    .withColumn("zip_code", getItem(col("location"), 1))
    .drop("location")
)
```

Correct answer: B

Explanation

This snippet perfectly aligns with the standard PySpark DataFrame API paradigms for data transformation:

- **`split(col("location"), " ")`**: The `split` function divides the string into an array of substrings based on the space delimiter.

- **Bracket Indexing `[0]` and `[1]`**: In PySpark, applying Python-style bracket notation to a Column containing an array extracts the element at that index (0 for the first element, 1 for the second).

- **withColumn(...)**: This creates a brand-new column while preserving the existing schema of df.
- **drop("location")**: This removes the original column as requested.

Question 4

“A centralized governance layer that manages metadata and access controls across workspaces and clouds. It organizes data using a three-level namespace (catalog.schema.table) and enables fine-grained permissions down to column level, while also providing audit logging and data lineage for compliance.”

Which of the following is being described in the above statement?

A. Databricks SQL

B. Delta Lake

C. Hive metastore

D. Unity Catalog

Correct answer: D

D. Unity Catalog

Explanation

The statement describes the exact architectural definition and core feature set of Databricks Unity Catalog:

- **Centralized governance across workspaces and clouds:** Unity Catalog acts as a unified governance solution for data and AI on the Databricks Lakehouse platform, operating across multiple Databricks workspaces and cloud providers (AWS, Azure, and GCP).

- **Three-level namespace (catalog.schema.table):** This is the standard data organization hierarchy introduced by Unity Catalog to reference data assets universally, replacing the older two-level namespace used by the traditional Hive metastore.

- **Fine-grained permissions and compliance:** It natively supports row-level and column-level security, along with automated data lineage (tracking how data flows from source to target) and built-in audit logging for regulatory compliance.

Why the other options are incorrect:

- **Hive metastore:** This is the legacy, two-level namespace (schema.table) metadata catalog. It does not natively support multi-workspace/multi-cloud centralized governance, column-level data lineage, or unified AI governance.

- **Databricks SQL:** This is the serverless data warehouse on Databricks that allows running SQL queries. While it uses Unity Catalog for governance, it is the compute and query layer, not the governance layer itself.

- **Delta Lake:** This is an open-source storage layer that brings ACID transactions and reliability to cloud data lakes. It manages data storage and file versions, not centralized user access controls or cross-workspace metadata governance.

Question 5

A data engineering team needs to run a Spark job that processes skewed data during a join operation, which may lead to performance delays. Adaptive Query Execution (AQE) is enabled in the cluster.

How does AQE help address data skew in this case?

- A. AQE automatically detects skewed partitions and splits them into smaller sub-partitions for parallel processing
- B. AQE removes skewed data from the dataset to dynamically mitigate the impact of data skew
- C. AQE converts all joins into Cartesian products to balance load and improve parallelism
- D. AQE disables partitioning entirely to improve execution efficiency while still preserving the benefits of distributed processing.

Correct answer: A

A. AQE automatically detects skewed partitions and splits them into smaller sub-partitions for parallel processing

Explanation

When Adaptive Query Execution (AQE) is enabled in Apache Spark, it monitors runtime statistics from completed stages. If it detects that a few partitions are significantly larger than the others (data skew), it dynamically intervenes before the join operation occurs.

Instead of letting a single task struggle with a massive, skewed partition while other tasks sit idle, AQE **splits the skewed partition into smaller sub-partitions**. It then joins each of these sub-partitions with the corresponding partition from the other side of the join. This allows Spark to process the heavy partition in parallel across multiple workers, preventing the "straggler task" bottleneck and drastically improving performance.

So, Adaptive Query Execution (AQE) helps identify and mitigate skewed join keys during execution.

Why the other options are incorrect:

- **AQE removes skewed data...**: Spark never drops or removes valid data from your dataset to fix performance; this would lead to incorrect business results.

- **AQE converts all joins into Cartesian products...**: Cartesian products ($O(N \times M)$ complexity) are incredibly expensive and slow. Spark tries to avoid them, not use them to fix performance.

- **AQE disables partitioning entirely...**: Disabling partitioning would break the core architecture of distributed computing in Spark.

Question 6

Which of the following statements best describes the usage of CREATE SCHEMA command ?

- A. It's used to infer and store schema in "cloudFiles.schemaLocation"
- B. It's used to merge the schema when writing data into a target table
- C. It's used to create a database
- D. It's used to create a table schema (columns names and datatype)

Correct answer: C

C. It's used to create a database

Explanation

CREATE SCHEMA is an alias for CREATE DATABASE statement. While usage of SCHEMA and DATABASE is interchangeable, SCHEMA is preferred.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 7

A data engineer configures a Declarative Automation Bundle (formerly Databricks Asset Bundle) and wants to push the assets to the defined target workspace.

Which Databricks CLI command can the data engineer use to achieve this goal?

A. databricks bundle deployment

B. databricks bundle validate

C. databricks bundle push

D. databricks bundle deploy

Correct answer: D

D. databricks bundle deploy

Explanation

In Databricks Asset Bundles (DABs), now known as Declarative Automation Bundles, the command for deploying your local project configuration and artifacts to a target workspace is `databricks bundle deploy`. This command compiles the bundle configuration artifacts, uploads them, and deploys the defined assets (such as jobs, pipelines, and notebooks) to the specific target workspace specified in your bundle's configuration file (e.g., `databricks.yml`).

Why the others are incorrect:

- **databricks bundle validate**: This checks the syntax and schema of your bundle configuration files to ensure they are valid, but it does not push or deploy anything to the workspace.
- **databricks bundle push**: This is not a valid Databricks CLI command for bundle management.
- **databricks bundle deployment**: This is not a valid command; the singular noun form is not used as an action verb in the Databricks CLI syntax.

Question 8

A data engineer has defined the following data quality constraint in a Lakeflow Spark Declarative Pipeline (formerly Delta Live Tables):

```
CONSTRAINT valid_id EXPECT (id IS NOT NULL) _____
```

Fill in the above blank so records violating this constraint cause the pipeline to fail.

A. ON VIOLATION FAIL

B. ON VIOLATION FAIL UPDATE

C. ON VIOLATION FAIL PIPELINE

D. There is no need to add ON VIOLATION clause. By default, records violating the constraint cause the pipeline to fail.

Correct answer: B

B. ON VIOLATION FAIL UPDATE

Explanation

With ON VIOLATION FAIL UPDATE, records that violate the expectation will cause the pipeline to fail. When a pipeline fails because of an expectation violation, you must fix the pipeline code to handle the invalid data correctly before re-running the pipeline.

* Please note that Delta Live Tables (DLT) has been recently renamed to Lakeflow Declarative Pipeline, however, the current exam version may still refer to it as Delta Live Tables.

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 9

A data engineer is preparing a Databricks asset bundle to define a job that executes a notebook located at `./src/my_notebook.py`. While writing the YAML configuration, the engineer drafts the following:

```
bundle:
  name: my_bundle

_____:
  jobs:
    my_job:
      name: my_notebook_job
      tasks:
        - task_key: test_task
          existing_cluster_id: 1234-911320-xyzwpm999
          notebook_task:
            notebook_path: './src/my_notebook.py'
```

To ensure the bundle is valid and deployable, which key should correctly replace the above blank?

- A. settings
- B. workflows
- C. pipelines
- D. resources

Correct answer: D

D. resources

Explanation

The resources section is where users define deployable objects, such as jobs, pipelines, notebooks, clusters, and more. These resources form the core of what is deployed when a bundle is applied to a target environment.

Options B, C, and D are incorrect:

- settings is not a valid section in the context of a Databricks Asset Bundle.
- pipelines allows you to create Lakeflow Declarative Pipelines (Delta Live Tables or DLT) pipelines
- workflows are not used in the **databricks.yml** structure.

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 10

A data engineering team manages Unity Catalog tables with predictive optimization enabled. They are unsure which operations are automatically performed on these tables as part of predictive optimization's automatic maintenance.

Which of the following operations is Not handled automatically by predictive optimization for enabled tables?

A. VACUUM

B. OPTIMIZE

C. ZORDER

D. ANALYZE

Correct answer: C

C. ZORDER

Explanation

Z-order indexing is not handled automatically by predictive optimization for Unity Catalog tables. While predictive optimization can automatically manage the OPTIMIZE, ANALYZE, and VACUUM tasks to maintain table performance, it does **not** execute ZORDER, and any Z-ordered files are ignored when predictive optimization runs.

Question 11

A data engineer currently uses an all-purpose cluster with auto-termination set to 6 hours to run a nightly incremental ETL job. The workloads complete at different times each night, ranging from 1-3 hours. They want to reduce compute costs while maintaining the same scheduling behavior.

Which approach should the data engineer use to reduce compute costs?

- A. Decrease the auto-termination setting on the all-purpose cluster to 3 hours
- B. Configure the all-purpose cluster to use smaller worker nodes
- C. Use a job compute cluster that automatically terminates after workload completion
- D. Use the AvailableNow trigger option to stop the stream processing automatically

Correct answer: C

- C. Use a job compute cluster that automatically terminates after workload completion

Explanation

In Databricks, **job clusters** are specifically designed for running automated workloads like nightly ETL jobs. They offer two major cost-saving advantages over all-purpose clusters:

- **Lower Billing Rate:** Job clusters are billed at a significantly lower Databricks Unit (DBU) rate compared to all-purpose clusters (which are intended for ad-hoc, interactive analysis).

- **Immediate Termination:** A job cluster is spun up automatically when the job starts and is completely terminated the exact moment the workload finishes. Because the engineer's workload variance spans from 1 to 3 hours, a job cluster eliminates 100% of the idle time waste without needing an arbitrary auto-termination timeout.

Why the other options are incorrect:

- **Decrease the auto-termination setting to 3 hours:** While this reduces the original 6-hour window, the cluster will still sit completely idle and run up costs for a fixed period after the job finishes. If the job finishes in 1 hour, you still pay for 2 hours of idle time before it shuts down. Furthermore, all-purpose DBU rates remain higher.

- **Configure the all-purpose cluster to use smaller worker nodes:** If the worker nodes are too small, the job might take significantly longer to run or fail due to Out-Of-Memory (OOM) errors, potentially increasing costs or missing SLAs. It doesn't solve the core issue of paying for idle time at an interactive rate.

- **Use the AvailableNow trigger option:** AvailableNow (or once=True in older syntax) is used in Structured Streaming to process all available data as a batch and then stop. While useful, it controls the software processing trigger, not the underlying infrastructure execution type. Running AvailableNow on an all-purpose cluster with a 6-hour timeout would still leave the cluster running idly after the stream finishes.

Question 12

A data engineer wants to review all changes made to a Delta table in the last 30 days, including transaction timestamps, operation types, and executing users. Which command should they use?

- A. `DESCRIBE HISTORY my_table`
- B. `DESCRIBE DETAIL my_table`
- C. `SELECT * FROM my_table VERSION AS OF <version_number>`
- D. `DESCRIBE EXTENDED my_table`

Correct answer: A

A. `DESCRIBE HISTORY my_table`

Explanation

In Delta Lake, the `DESCRIBE HISTORY` command provides a complete audit trail of a table. It returns an ordered list of all operations performed on the table, including:

- **Transaction Timestamps:** When the change occurred (timestamp).
- **Operation Types:** What action was taken, such as `WRITE`, `MERGE`, `OPTIMIZE`, or `STREAMING UPDATE` (operation).
- **Executing Users:** The identity of the user or service principal that ran the operation (userId and userName).
- **Operation Parameters & Metrics:** Specific details about what changed, like the number of rows inserted, updated, or deleted (operationParameters and operationMetrics).

By default, Delta Lake retains table history for 30 days (controlled by the table property `delta.logRetentionDuration`), making this command perfect for reviewing changes over the last month.

Why the other options are incorrect:

- **DESCRIBE EXTENDED my_table:** This command displays the basic schema definition of the table alongside extended metadata (such as the database, table type, and properties), but it does not show transaction or operational history.
- **DESCRIBE DETAIL my_table:** This provides high-level information about the current state of the table, such as its size on disk, number of files, format, and partition columns, but lacks transaction history.
- **SELECT * FROM my_table VERSION AS OF <version_number>:** This is used for **Time Travel**.

While it allows you to query a snapshot of the data exactly as it looked at a specific version, it won't give you a breakdown list of transaction metadata, operations, or executing users.

Question 13

A data engineer is validating incoming transaction data in a Databricks SQL pipeline. Some records contain unexpected symbols in numeric fields. The engineer runs the following query:

```
SELECT INT("100$")
```

Which of the following describes the result of this query?

- A. 100
- B. \$100
- C. NULL
- D. An error is thrown

Correct answer: D

D. An error is thrown

Explanation

By default, if you use the `INT` function on a string that contains characters that are not part of the target numerical type (such as the trailing `$` symbol in `'100$'`), the query will fail immediately and throw a runtime exception: **[CAST_INVALID_INPUT]**.

Question 14

A data engineering team is processing a large-scale ETL pipeline that involves joining multiple large datasets, each containing hundreds of columns and billions of records. During the join phase, they notice that the Spark executors are repeatedly spilling data to disk, and performance significantly degrades due to excessive shuffling.

What type of resource optimization should the team prioritize to improve the performance of this job?

- A. GPU Optimized
- B. Storage Optimized
- C. Compute Optimized
- D. Memory Optimized

Correct answer: D

D. Memory Optimized

Explanation

Spark joins, particularly when dealing with large datasets or complex join conditions, can be highly memory-intensive. This is due to the need to shuffle and potentially store data in memory during the join operation.

A memory-optimized resources ensures that more in-memory operations can be completed without disk spills, thereby significantly reducing shuffle overhead and improving performance.

Worker type ⓘ

Standard_D4ds_v5	16 GB Memory, 4 Cores	▼
General purpose		
Standard_DS3_v2	14 GB Memory, 4 Cores	
Standard_DS4_v2	28 GB Memory, 8 Cores	
Standard_DS5_v2	56 GB Memory, 16 Cores	
Memory optimized		
Standard_DS12_v2	28 GB Memory, 4 Cores	
Standard_DS13_v2	56 GB Memory, 8 Cores	
Storage optimized		
Standard_L8s_v3	64 GB Memory, 8 Cores	
Standard_L16s_v3	128 GB Memory, 16 Cores	
Standard_L32s_v3	256 GB Memory, 32 Cores	
Compute optimized		
▶ Advanced options		

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 15

A data engineering team needs to ingest clickstream events from an Apache Kafka topic into Databricks. The system receives 10,000 events per second, and the data must be processed with a latency of under 60 seconds.

Which ingestion approach is most appropriate?

- A. Batch ingestion using scheduled Spark jobs every minute
- B. Structured Streaming from Kafka into Databricks
- C. Full table reload from Kafka every 24 hours
- D. Incremental ingestion using a Lakeflow managed connector for Kafka

Correct answer: B

B. Structured Streaming from Kafka into Databricks

Explanation

The scenario specifies that the system must process data with a latency of **under 60 seconds** at a scale of 10,000 events per second. Spark Structured Streaming is natively built for low-latency, real-time, continuous stream processing. It utilizes a micro-batch or continuous processing engine that comfortably reduces latency down to sub-second or few-second intervals, making it ideal for clickstream tracking.

Apache Spark features native, highly optimized integration connectors for Apache Kafka. It allows you to query Kafka topics as unbound data streams, maintaining end-to-end exactly-once processing guarantees and high fault tolerance.

```
df = (spark.readStream
      .format("kafka")
      .option("kafka.bootstrap.servers", "<server:ip>")
      .option("subscribe", "<topic>")
      .option("startingOffsets", "latest")
      .load()
    )
```

Why the other options are incorrect:

- **Batch ingestion using scheduled Spark jobs every minute:** While a 60-second schedule sounds like it meets the 60-second SLA, classic batch scheduling introduces major overhead (spinning up/down resources or scheduling delay). If processing takes more than a few seconds, or if there is a minor spike in the 10,000 events/sec volume, the jobs will cascade, backpressure will mount, and you will quickly violate the sub-60-second latency guarantee.

- **Incremental ingestion using a Lakeflow managed connector for Kafka:** Databricks Lakeflow Connect is built for automated, turn-key ingestion of applications and databases (such as Salesforce, SQL Server, or Oracle) into the Lakehouse. High-throughput, real-time event streaming systems like

Kafka are natively solved via direct Structured Streaming pipelines.

- **Full table reload from Kafka every 24 hours:** This represents a 24-hour latency, which fundamentally violates the <60 seconds real-time requirement.

Question 16

In Databricks Jobs, which of the following approaches can a data engineer use to configure a linear dependency between **Task A** and **Task B** ?

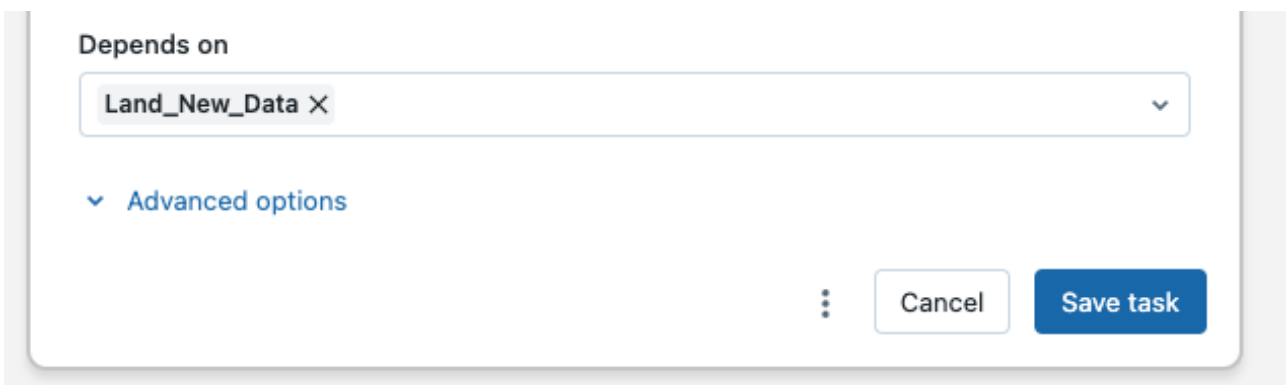
- A. They can select the Task A in the **Depends On** field of the Task B configuration
- B. They can visually drag and drop an arrow from Task A to Task B in the Job canvas
- C. They can configure the dependency at the notebook level using the **dbutils.jobs** utility
- D. They can assign Task A an Order number of 1, and assign Task B an Order number of 2

Correct answer: A

A. They can select the Task A in the Depends On field of the Task B configuration

Explanation

You can define the order of execution of tasks in a job using the **Depends on** dropdown menu. You can set this field to one or more tasks in the job.



The screenshot shows a configuration window for a task in Databricks Jobs. At the top, the title 'Depends on' is displayed. Below it is a dropdown menu with the text 'Land_New_Data X' and a small 'X' icon to its right. Below the dropdown is a link labeled 'Advanced options' with a downward-pointing chevron. At the bottom right of the window are three buttons: a vertical ellipsis (three dots), a 'Cancel' button, and a 'Save task' button.

* Please note that Databricks Jobs has been recently renamed to Lakeflow Jobs, however, the current exam version may still refer to it as Databricks Jobs.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 17

Which of the following technologies is used by Auto Loader to load data incrementally?

- A. COPY INTO
- B. Multi-hop architecture
- C. DEEP CLONE
- D. Spark Structured Streaming

Correct answer: D

D. Spark Structured Streaming

Explanation

Auto Loader is based on Spark Structured Streaming. It provides a Structured Streaming source called cloudFiles.

Reference: <https://docs.databricks.com/ingestion/auto-loader/index.html>

Study materials from our exam preparation course on Udemy:

- [Lecture](#)
- [Hands-on](#)

Question 18

A data engineer wants to increase the cluster size of an existing Databricks SQL warehouse.

Which of the following is the benefit of increasing the cluster size of Databricks SQL warehouses ?

- A. The cluster size of SQL warehouses is not configurable. Instead, they can increase the number of clusters
- B. Reduces cost since large clusters use Spot instances
- C. Speeds up the start up time of the SQL warehouse
- D. Reduces the latency of the queries execution

Correct answer: D

- D. Reduces the latency of the queries execution

Explanation

Cluster Size represents the number of cluster workers and size of compute resources available to run your queries and dashboards. To reduce query latency, you can increase the cluster size.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 19

Which of the following operations can a data engineer use to update a Git folder from its remote Git repository?

A. Commit

B. Clone

C. Pull

D. Push

Correct answer: C

C. Pull

Explanation

The git Pull operation is used to fetch and download content from a remote repository and immediately update the local repository to match that content.

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 20

A data engineer is working with a table `products` that has the following schema:

`product_id` STRING,

`updated_at` TIMESTAMP,

`details` STRUCT<name: STRING, category: STRING, pricing: STRUCT<base_price: DOUBLE, discount: DOUBLE>>,

They want to flatten all fields inside `details` so that `name`, `category`, and the nested pricing fields become root-level columns.

Which SQL query correctly achieves this goal?

A.

```
SELECT *  
FROM products;
```

B.

```
SELECT  
  product_id, updated_at, details.name, details.category,  
  details.pricing.base_price, details.pricing.discount  
FROM products;
```

C.

```
SELECT  
  product_id, updated_at, details.*  
FROM products;
```

D.

```
SELECT  
  product_id, updated_at, EXPLODE(details)  
FROM products;
```

Correct answer: B

Explanation

In Databricks SQL, a **STRUCT** data type represents a collection of named fields (essentially a row nested inside a column). To extract a field from a **STRUCT** and pull it up to the root level, you use **dot notation** (`column.field`).

- **`details.name`** extracts the string `name` field from the root `details` struct.

- **`details.pricing.base_price`** demonstrates accessing deeply nested structures. Since `pricing` is itself a **STRUCT** nested within `details`, you daisy-chain the dot operators (`column.nested_struct.field`) to cleanly surface the innermost values as root-level columns.

Why the other options are incorrect:

SELECT * FROM products;

- **What it does:** This returns every column at the root level of the table as-is.

- **Why it's wrong:** It retains the complex types without breaking them apart. You will still end up with exactly three columns: product_id, updated_at, and the parent object details preserving its nested hierarchy.

SELECT product_id, updated_at, details.* FROM products;

- **What it does:** The star modifier (.*) can expand the fields of a first-level STRUCT.

- **Why it's wrong:** While this expression will partially flatten the data by extracting name, category, and pricing, it will **not** flatten the nested pricing struct. The pricing column will remain a nested struct data type (STRUCT<base_price: DOUBLE, discount: DOUBLE>) rather than two independent root-level columns. To fully flatten the table using star notation, you would have to write details.name, details.category, details.pricing.*.

SELECT product_id, updated_at, EXPLODE(details) FROM products;

- **What it does:** The EXPLODE() function is designed exclusively for collection types like **ARRAY** (unnesting items into separate rows) or **MAP** (splitting key-value pairs).

- **Why it's wrong:** Passing a STRUCT data type into EXPLODE() will result in a compilation error (AnalysisException). Structs have a static, predefined schema and cannot be processed like standard iterative collections.

Question 21

A data engineer wants to incrementally ingest sales transaction data from multiple enterprise sources (Salesforce, SQL database, Kafka stream, and Azure Data Lake Storage) into Unity Catalog tables.

Which configuration should the data engineer use to meet this requirement?

- A. Single managed connector for all sources with CDC
- B. Single standard connector for all sources with CDC
- C. Separate connectors per source with CDC
- D. Single unified Lakeflow connector for all sources with CDC

Correct answer: C

- C. Separate connectors per source with CDC

Explanation

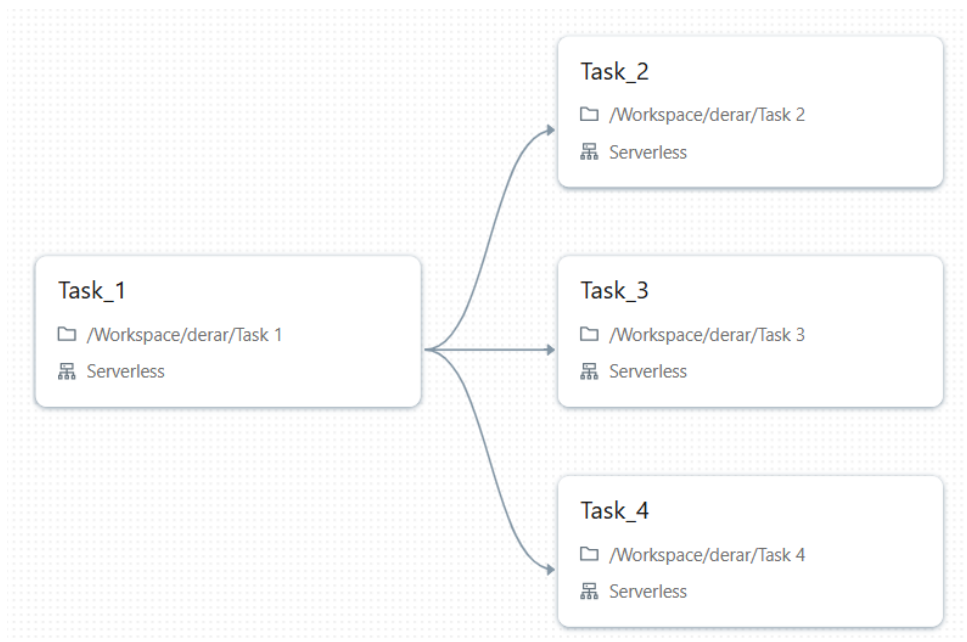
In **Lakeflow Connect**, connectors are specialized by data source type rather than packaged as a single generic "unified" connector for completely distinct technologies (SaaS vs. Relational Database vs. Event Stream vs. Cloud Storage). The data engineer must configure distinct ingestion pipelines for each respective source category:

- **SaaS Connectors:** Used for Salesforce (utilizing cursor-based incremental tracking or change capture APIs).
- **Database Connectors (CDC):** Used for relational databases like a SQL database (leveraging native Change Data Capture mechanisms like Microsoft CT/CDC or Debezium protocols).
- **Standard Connectors:** Used for streaming event message buses like Kafka and cloud object storages like Azure Data Lake Storage (ADLS, via Auto Loader/Streaming Tables).

Each of these data sources requires its own unique authentication methods, gateway configurations, and change tracking mechanisms under Unity Catalog governance. Therefore, separate connectors must be configured per source to scale and process incremental data into Unity Catalog effectively.

Question 22

A data engineer maintains the following Lakflow Job:



They want to add a new task, Task 5, to run after the completion of all the parallel tasks (Tasks 2, 3, and 4).

Which task configuration meets this requirement?

- A. Add Task 5 without any dependency but with condition run_if: ALL_DONE
- B. Configure Task 5 in a separate job and schedule it to run after the current job completes.
- C. Set Task 5 to depend only on Task 4, which runs after Tasks 2 and 3 in order.
- D. Set Task 5 to depend on all three parallel tasks using fan-in control flow

Correct answer: D

- D. Set Task 5 to depend on all three parallel tasks using fan-in control flow

Explanation

In **Databricks Lakeflow Jobs**, a job's structure is visually represented as a Directed Acyclic Graph (DAG).

- Fan-out/Fan-in Patterns:

- The current configuration shows a **fan-out** pattern, where multiple downstream tasks (Task_2, Task_3, and Task_4) run in parallel after the completion of a single upstream root task (Task_1).

- To execute a new task (Task_5) only after all parallel steps are completed, you need to configure a **fan-in** pattern. This means modifying Task_5 so that its Depends on property includes all three parallel tasks (Task_2, Task_3, and Task_4).

- Why other options are incorrect:

- Add Task 5 without any dependency but with condition run_if: ALL_DONE: If a task has no explicitly defined upstream dependencies, it will execute immediately as a root node alongside Task_1 when the job starts, ignoring the run_if logic.

- Set Task 5 to depend only on Task 4...: Task_2, Task_3, and Task_4 are entirely parallel and independent of each other (indicated by separate arrows sprouting directly from Task_1). Task_4 does not run after Task_2 and Task_3 sequentially.

- Configure Task 5 in a separate job...: While technically possible using a separate pipeline trigger, it adds unnecessary operational complexity and loses unified lineage tracking within the same Lakeflow DAG configuration.

Question 23

A data engineer schedules a notebook as a Databricks Job to process daily confidential data. They are concerned about where the notebook code actually runs when the job is triggered.

Which of the following explains where the execution of this job occurs?

- A. In the workspace storage bucket
- B. In Unity Catalog during query planning
- C. In the control plane where the notebook is stored
- D. In the compute plane on a cluster running Spark

Correct answer: D

D. In the compute plane on a cluster running Spark

Explanation

In the compute plane on a cluster running Spark

Databricks splits its architecture into two core layers: the **Control Plane** and the **Compute Plane** (historically called the Data Plane). While the notebook's code template and metadata reside in the control plane, the **actual execution of data processing commands occurs entirely within the compute plane** on assigned compute resources (clusters running Apache Spark). This ensures that your actual confidential raw data is processed within a secure network boundary (either classic or serverless) and never exposes your raw data streams to Databricks' core control systems.

Why the other options are incorrect:

- In the control plane where the notebook is stored

Why it is wrong: The control plane houses Databricks' backend services, the web application UI, notebook source code at rest, and administrative configurations. It does not possess the Spark computational frameworks or resources to crunch your data, nor does it ever directly interact with or process your raw confidential data payloads.

- In Unity Catalog during query planning

Why it is wrong: Unity Catalog is Databricks' centralized data governance tool. It handles metadata, controls table/file permissions, tracks data lineage, and determines if you have the right to query something during planning. It does not serve as the execution engine that processes the notebook code.

- In the workspace storage bucket

Why it is wrong: The workspace storage bucket (such as an AWS S3 bucket or Azure DBFS root storage) is used strictly for storing persistent internal workspace systems data, libraries, logs, or your notebook files at rest. Storage buckets do not possess active CPU or RAM processing capabilities to run code.

Question 24

A junior data engineer usually uses `INSERT INTO` command to write data into a Delta table. A senior data engineer suggested using another command that avoids writing of duplicate records.

Which of the following commands is the one suggested by the senior data engineer ?

- A. `MERGE INTO`
- B. `COPY INTO`
- C. `INSERT OR OVERWRITE`
- D. `UPDATE`

Correct answer: A

A. `MERGE INTO`

Explanation

`MERGE INTO` allows to merge a set of updates, insertions, and deletions based on a source table into a target Delta table. With `MERGE INTO`, you can avoid inserting the duplicate records when writing into Delta tables.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 25

A data engineer is analyzing a Spark job via the Spark UI. They have the following summary metrics for 27 completed tasks in a particular stage

Summary Metrics for 27 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	1.6 s	2 s	2 s	3 s	30 s
GC Time	20.0 ms	32.0 ms	44.0 ms	51.0 ms	1.9 s
Input Size / Records	78 KiB / 16	179 KiB / 49	183.9 KiB / 55	186.7 KiB / 67	803 KiB / 276
Shuffle Write Size / Records	1.4 KiB / 16	6.1 KiB / 49	6.9 KiB / 55	8.1 KiB / 67	300.8 KiB / 276

Which conclusion can the data engineer draw from the above statistics ?

- A. Number of tasks are operating over empty or near empty partitions
- B. Number of tasks are operating over partitions with larger skewed amounts of data.
- C. All task are operating over partitions with even amounts of data
- D. All tasks are operating over partitions with larger skewed amounts of data.

Correct answer: B

B. Number of tasks are operating over partitions with larger skewed amounts of data.

Explanation

Usually, if your computation was completely symmetric across tasks, you would see all of the statistics clustered tightly around the 50th percentile value.

Here, the “Max” metrics task took 10x the time and read about 5x the data of the 75th-percentile task. This suggests a number of “straggler” tasks that operate over partitions with larger skewed amounts of data.

Question 26

A junior data engineer has been tasked with implementing data quality validation in a Lakeflow Spark Declarative Pipeline (SDP). They added several expectation functions to ensure that incoming datasets meet certain criteria before being processed further. After the junior engineer submitted a pull request, a senior data engineer began reviewing the code and noticed that one of the function calls used for validation was not correct.

As part of the review, the senior engineer wants to ensure that all expectation functions used in the pipeline are valid according to Databricks documentation.

Which of the following function calls is Not a valid expectation function in Lakeflow Declarative Pipelines?

A. `expect_or_drop(...)`

B. `expect_or_warn(...)`

C. `expect(...)`

D. `expect_or_fail(...)`

Correct answer: B

B. `expect_or_warn(...)`

Explanation

`expect_or_warn()` is not a supported expectation function in Lakeflow Spark Declarative Pipelines (SDP), formerly known as Delta Live Tables.

SDP supports the following expectation functions:

- `expect()`: it writes invalid rows to the target (warning semantics)
- `expect_or_drop()`: drops invalid rows before writing to the target.
- `expect_or_fail()`: fails the update if violation occurs

Question 27

A data engineer needs to determine whether to use Auto Loader or COPY INTO command in order to load input data files incrementally.

In which of the following scenarios should the data engineer use Auto Loader over COPY INTO command ?

- A. If they are going to load a subset of re-uploaded files
- B. If they are going to ingest few number of files in the order of thousands
- C. If they are going to ingest files in the order of millions or more over time
- D. If the data schema is not going to evolve frequently

Correct answer: C

- C. If they are going to ingest files in the order of millions or more over time

Explanation

Here are a few things to consider when choosing between Auto Loader and COPY INTO command:

- If you're going to ingest files in the order of thousands, you can use COPY INTO. If you are expecting files in the order of millions or more over time, use Auto Loader.
- If your data schema is going to evolve frequently, Auto Loader provides better primitives around schema inference and evolution.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)
- [Hands-on](#)

Question 28

A data engineer has a DataFrame, `employeesDf`, with a column named `salary` and wants to permanently rename this column to `base_salary` for all future operations on `employeesDf`.

Which PySpark code fragment meets this requirement?

A. `employeesDf = employeesDf.select(col("*"), col("salary").alias("base_salary"))`

B. `employeesDf = employeesDf.withColumnRenamed("salary", "base_salary")`

C. `employeesDf = employeesDf.select(col("salary").alias("base_salary"))`

D. `employeesDf = employeesDf.withColumn("salary", "base_salary")`

Correct answer: B

B. `employeesDf = employeesDf.withColumnRenamed("salary", "base_salary")`

Explanation

In Apache Spark, DataFrames are **immutable**, meaning their structure cannot be altered in place. To make a "permanent" change for all future operations, you must apply the transformation and reassign the resulting DataFrame back to the original variable name (`employeesDf`).

- **`withColumnRenamed(existingName, newName)`** is the dedicated PySpark method designed specifically to rename an existing column without modifying the rest of the schema.

- The assignment operator (`=`) rebinds the variable `employeesDf` to the new DataFrame structure containing the renamed column.

Why the other options are incorrect:

- `employeesDf = employeesDf.withColumn("salary", "base_salary")`: This would attempt to add or replace a column named "salary" with the literal string value "base_salary", rather than renaming it.

- `employeesDf = employeesDf.select(col("salary").alias("base_salary"))`: While this renames the column, it drops all other columns from the DataFrame, leaving only `base_salary`.

- `employeesDf = employeesDf.select(col("*"), col("salary").alias("base_salary"))`: This creates a duplicate column. The original "salary" column will still exist alongside a new column named "base_salary".

Question 29

A data engineer is configuring a Databricks Auto Loader stream to ingest JSON data from an S3 bucket. The pipeline should fail when new columns are detected in the incoming data, but those new columns should still be added to the schema so that subsequent runs can resume successfully with the updated schema. Existing columns must retain their data types.

```
spark.readStream \
  .format("cloudFiles") \
  .option("cloudFiles.format", "json") \
  .option("cloudFiles.schemaLocation", "s3://checkpoints/orders")
  .option("cloudFiles.schemaEvolutionMode", "_____") \
  .load("s3://shop/raw/orders/json/") \
  .writeStream \
  .option("checkpointLocation", "s3://checkpoints/orders") \
  .start("orders_table")
```

Which option correctly fills in the blank to meet the specified requirement ?

- A. addNewColumns
- B. failOnNewColumns
- C. none
- D. rescue

Correct answer: A

A. addNewColumns

Explanation

The addNewColumns mode is the default schema evolution behavior in Auto Loader. In this mode, when a new column is detected, the stream fails, but the new column is added to the schema. This allows the job to be restarted and continue processing with the updated schema. Importantly, existing columns' data types are not changed.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 30

A data engineer noticed that there are unused data files in the directory of a Delta table. They executed the VACUUM command on this table; however, only some of those unused data files have been deleted.

Which of the following could explain why only some of the unused data files have been deleted after running the VACUUM command ?

- A.** The deleted data files were larger than the default size threshold. While the remaining files are smaller than the default size threshold and can not be deleted.
- B.** The deleted data files were smaller than the default size threshold. While the remaining files are larger than the default size threshold and can not be deleted.
- C.** The deleted data files were older than the default retention threshold. While the remaining files are newer than the default retention threshold and can not be deleted.
- D.** The deleted data files were newer than the default retention threshold. While the remaining files are older than the default retention threshold and can not be deleted.

Correct answer: C

C. The deleted data files were older than the default retention threshold. While the remaining files are newer than the default retention threshold and can not be deleted.

Explanation

Running the VACUUM command on a Delta table deletes the unused data files older than a specified data retention period. Unused files newer than the default retention threshold are kept untouched.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 31

For production jobs, which of the following cluster types is recommended to use?

- A. On-premises clusters
- B. All-purpose clusters
- C. Job clusters
- D. Production clusters

Correct answer: C

C. Job clusters

Explanation

Job Clusters are dedicated clusters for a job or task run. A job cluster auto terminates once the job is completed, which saves cost compared to all-purpose clusters.

In addition, Databricks recommends using job clusters in production so that each job runs in a fully isolated environment.

* Please note that Databricks Jobs has been recently renamed to Lakeflow Jobs, however, the current exam version may still refer to it as Databricks Jobs.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 32

Which scenario is best suited for using spot instances in Databricks?

- A. Real-time stream processing with strict SLA requirements
- B. Transactional workloads with high concurrency demands
- C. Interactive data analysis by business analysts
- D. Non-critical batch processing jobs with retry capability

Correct answer: D

D. Non-critical batch processing jobs with retry capability

Explanation

Spot instances are ideal for stateless, fault-tolerant workloads-such as batch processing jobs that can be retried without significant data loss or user impact. These jobs are often time-flexible and can tolerate interruptions, making them a good match for spot instances.

Worker type ⓘ

Standard_D4ds_v5

16 GB Memory, 4 Cores



Min workers

2



Max workers

8



Spot instances ⓘ

In contrast, use cases that demand low latency, real-time processing, or guaranteed uptime-such as interactive queries or transactional systems-are generally unsuited for spot instances due to the inherent risk of unexpected termination.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 33

A financial services company wants to migrate nightly account data from an on-premises SQL Server database into a governed analytics platform. The ingested records must be stored in a managed Unity Catalog table so analysts can query the data immediately using SQL dashboards, while maintaining automatic lineage tracking for compliance audits.

Which implementation best satisfies these requirements?

- A. Use a local Python script with Pandas and upload Parquet files to Unity Catalog Volume
- B. Read data using Spark JDBC and write directly into a Unity Catalog managed table
- C. Extract data through ODBC into temporary files before manually registering tables
- D. Export CSV files to object storage and load them later with a separate ETL pipeline

Correct answer: B

B. Read data using Spark JDBC and write directly into a Unity Catalog managed table

Explanation

The implementation that best satisfies these requirements is to read data using Spark JDBC and write directly into a Unity Catalog managed table.

```
df = (
    spark.read
        .format("jdbc")
        .option("url", "jdbc:postgresql://host:5432/db")
        .option("dbtable", "schema.source_table")
        .option("user", "username")
        .option("password", "password")
        .load()
)

df.write.mode("overwrite").saveAsTable("catalog.schema.target_table")
```

Unity Catalog automatically captures runtime lineage for any data read or written using Apache Spark DataFrame API operations within Databricks. By reading directly via Spark JDBC and writing straight into a Unity Catalog managed table, the platform seamlessly tracks the source-to-target relationship for compliance audits.

Writing data directly into a managed table makes it immediately available for analysts to query through SQL dashboards or Databricks SQL warehouses, while Unity Catalog fully governs the managed table by handling both metadata and underlying storage to ensure strong access control, auditing, and centralized data management.

Why the other options are incorrect:

- **Export CSV files to object storage...** This adds unnecessary architectural complexity and introduces latency, meaning analysts cannot query the data immediately. It also risks breaking end-to-end automated lineage tracking if the file generation and loading stages are disjointed.

- **Use a local Python script with Pandas...** Pandas runs on a single node (local machine) and is not designed for robust, enterprise-level big data ingestion.

Extract data through ODBC into temporary files... Manual table registration and the creation of temporary files introduce significant operational friction, prevent immediate querying, and completely bypass Unity Catalog's automated runtime lineage tracking.

Question 34

Which of the following statements is **Not** true about CTAS statements ?

- A. CTAS statements stand for CREATE TABLE _ AS SELECT statement
- B. CTAS statements automatically infer schema information from query results
- C. CTAS statements support manual schema declaration
- D. With CTAS statements, data will be inserted during the table creation

Correct answer: C

- C. CTAS statements support manual schema declaration

Explanation

CREATE TABLE AS SELECT statements, or CTAS statements create and populate Delta tables using the output of a SELECT query. CTAS statements automatically infer schema information from query results and do not support manual schema declaration.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 35

A data engineer has been tasked with granting the service principal with Application ID 'fe7bcf95-ab87-4dce-a2fd-8c55f8158a02' a read-only access to the enterprise.reporting.transactions table.

Which SQL commands will correctly grant access with the least privileges?

A.

```
GRANT SELECT ON TABLE enterprise.reporting.transactions TO  
`fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;  
GRANT USE SCHEMA ON SCHEMA enterprise.reporting TO  
`fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;
```

B.

```
GRANT SELECT ON TABLE enterprise.reporting.transactions TO  
`fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;  
GRANT USE CATALOG ON CATALOG enterprise TO `fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;
```

C.

```
GRANT SELECT ON TABLE enterprise.reporting.transactions TO  
`fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;  
GRANT USE SCHEMA ON SCHEMA enterprise.reporting TO  
`fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;  
GRANT USE CATALOG ON CATALOG enterprise TO `fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;
```

D.

```
GRANT SELECT ON TABLE enterprise.reporting.transactions TO  
`fe7bcf95-ab87-4dce-a2fd-8c55f8158a02`;
```

Correct answer: C

Explanation

In Unity Catalog, a securable object operates within a strict hierarchy (Catalog -> Schema -> Object). To perform an action on any object, a principal (user, group, or SP) must not only have the privilege for that specific object but also the permission to traverse its parent containers.

To grant a principal read-only (SELECT) access to a specific table with the least privileges, you must explicitly grant the following hierarchical permissions:

- **USE CATALOG** on the parent catalog (enterprise): This allows the principal to traverse the catalog.
- **USE SCHEMA** on the parent schema (enterprise.reporting): This allows the principal to traverse the schema.
- **SELECT** on the target table (enterprise.reporting.transactions): This allows the principal to read the data from the table.

Without the `USE CATALOG` and `USE SCHEMA` privileges, the service principal will not be able to see or access the table, even if they have direct `SELECT` permissions on it.

Question 36

A finance analytics team manages a Delta Lake table in Unity Catalog called “transactions” with columns: id, amount, region, and account_manager. They want to apply row filtering on this table so that:

Finance team members can see all transactions, while

Other users can only see records from the US region.

Which of the following user-defined functions help achieve this?

A.

```
CREATE FUNCTION us_filter(region STRING)
RETURN CASE
    WHEN IS_ACCOUNT_GROUP_MEMBER('finance_team') THEN true
    ELSE region END
```

B.

```
CREATE FUNCTION us_filter(region STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('finance_team'), region='US', true);
```

C.

```
CREATE FUNCTION us_filter(region STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('finance_team'), true, region='US');
```

D.

```
CREATE FUNCTION us_filter(region STRING)
RETURN CASE
    WHEN IS_ACCOUNT_GROUP_MEMBER('finance_team') THEN region='US'
    ELSE true END
```

Correct answer: C

Explanation

The correct user-defined function is:

```
CREATE FUNCTION us_filter(region STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('finance_team'), true, region='US');
```

This function properly enforces the desired row-level security logic. It first checks whether the current user belongs to the **finance_team** group using `IS_ACCOUNT_GROUP_MEMBER('finance_team')`. If the user is a member, the function returns true, allowing them to see all transaction records without restriction. For all other users who are not part of the finance team, the function evaluates whether the region value equals 'US', thereby restricting their visibility to only U.S. transactions.

Question 37

A junior data engineer creates a Databricks job with 15 notebook tasks, each performing the same data validation logic on 15 different tables. Each task depends on the completion of the previous one, making the workflow long and difficult to maintain.

What would be a more efficient and scalable solution for this use case?

- A. Use a foreach task to run the same validation notebook for each table in parallel, passing the table name as a parameter
- B. Configure the 15 notebook tasks to run in parallel, each with a separate cluster configuration
- C. Combine all table validations into one large notebook and loop through all tables sequentially
- D. Schedule 15 separate jobs instead of having multiple tasks in one job

Correct answer: A

A. Use a foreach task to run the same validation notebook for each table in parallel, passing the table name as a parameter

Explanation

A more efficient and scalable solution in this scenario is to use a For Each task. The For Each task allows you to run a nested task in a loop, passing different parameters to each iteration. In this case, the data engineer can pass each table name as a parameter, running the same validation notebook for all tables. This approach reduces maintenance overhead, and allows the validations to run concurrently to avoid sequential dependencies, making the workflow faster and easier to manage.

Question 38

Which part of the Databricks platform can a data engineer use to grant and revoke permissions on securable objects for principals?

- A. Account console
- B. Data Studio
- C. Catalog Explorer
- D. Workspace Admin Console

Correct answer: C

C. Catalog Explorer

Explanation

Catalog Explorer allows you to manage data object permissions. This includes granting and revoking privileges on securable objects (such as tables and databases) for principals (users, groups of users, or service principals).

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 39

A data engineer wants to use Declarative Automation Bundles (formerly known as Databricks Asset Bundles) to deploy jobs into the dev workspace via a PAT token from their local machine.

Which TWO environment variables should the data engineer set in their shell environment to configure Databricks Unified Authentication?

- A. DATABRICKS_HOST
- B. DATABRICKS_CLIENT_SECRET
- C. DATABRICKS_CLIENT_ID
- D. DATABRICKS_WORKSPACE
- E. DATABRICKS_TOKEN

Correct answers: A, E

- A. DATABRICKS_HOST
- E. DATABRICKS_TOKEN

Explanation

To configure Databricks Unified Authentication using a Personal Access Token (PAT) for local deployment with Declarative Automation Bundles (DABs), you need to set the following two environment variables:

- **DATABRICKS_HOST**
- **DATABRICKS_TOKEN**

Setting these two variables in your local shell enables the Databricks CLI (which drives Declarative Automation Bundles) to automatically detect and authenticate your deployment session against the targeted development workspace without needing an explicit configuration file.

IMPORTANT: Where possible, Databricks recommends using OAuth instead of PATs for user account authentication because OAuth provides stronger security.

Question 40

A data analyst works with two PySpark DataFrames: **teachers_df** and **courses_df**, both containing a `teacher_id` column. They need a DataFrame of teachers who do not teach any courses.

Which PySpark code achieves this requirement?

- A. `joined_df = teachers_df.join(courses_df, "teacher_id", "left")`
- B. `joined_df = teachers_df.join(courses_df, "teacher_id", "left_semi")`
- C. `joined_df = teachers_df.join(courses_df, "teacher_id", "left_anti")`
- D. `joined_df = teachers_df.join(courses_df, "teacher_id", "full")`

Correct answer: C

C. `joined_df = teachers_df.join(courses_df, "teacher_id", "left_anti")`

Explanation

An **anti join** (specifically a `left_anti` join in PySpark) returns values from the left DataFrame (`teachers_df`) that **do not have a match** in the right DataFrame (`courses_df`).

Since the analyst wants a list of teachers who do not teach any courses, the anti join automatically filters out any `teacher_id` that appears in the `courses` table, leaving exactly the subset of teachers without a course assignment.

How it compares to the other options

- **left_semi**: This is the exact opposite of an anti join. It returns only the rows from the left DataFrame that do have a matching record in the right DataFrame (i.e., teachers who are teaching at least one course).
- **left**: A standard left outer join returns all teachers regardless of whether they teach a course. For teachers without a course, the columns originating from `courses_df` would simply contain null values. (To make this option work, you would need an additional `.filter()` or `.where()` clause checking for those nulls).
- **full**: A full outer join returns all records from both DataFrames, matching them where possible and filling with null where no match exists.

Question 41

In the Medallion Architecture, which of the following statements best describes the Bronze layer?

- A. It maintains raw data ingested from various sources
- B. It represents a filtered, cleaned, and enriched version of data
- C. It provides business-level aggregated version of data
- D. It maintains data that powers analytics, machine learning, and production applications

Correct answer: A

- A. It maintains raw data ingested from various sources

Explanation

Bronze tables contain data in its rawest format ingested from various sources (e.g., JSON files, Operational Databases, Kafka stream, ...)

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 42

A data engineer defines the following Unity Catalog ABAC policy to centrally apply column masking using the `mask_credit_card(cc STRING)` UDF.

```
CREATE POLICY banking_policy ON SCHEMA bank.safebox
COLUMN MASK bank.safebox.mask_credit_card
TO financial_analysts EXCEPT admins
FOR TABLES
_____
ON COLUMN cc;
```

They need to mask every column tagged `pii=credit_card` across all tables in the schema `bank.safebox`. Users in the group `financial_analysts` should see masked values, while users in the group `admins` should see unmasked values.

Which option correctly fills in the blank to meet the specified requirement?

- A. `MATCH COLUMNS hasTagValue('pii', 'cc') AS credit_card`
- B. `MATCH COLUMNS hasTag('pii') AS credit_card`
- C. `MATCH COLUMNS hasTagValue('pii') AS credit_card`
- D. `MATCH COLUMNS hasTagValue('pii','credit_card') AS cc`

Correct answer: D

D. `MATCH COLUMNS hasTagValue('pii','credit_card') AS cc`

Explanation

In Databricks Unity Catalog's Attribute-Based Access Control (ABAC) policies, you use specialized metadata functions inside the `MATCH COLUMNS` clause to identify which columns the policy targets:

- **Identifying Tag Keys and Values:** To check if a column has a specific tag key (`pii`) with a specific tag value (`credit_card`), you must use the `hasTagValue('tag_key', 'tag_value')` function (or its recommended snake_case equivalent `has_tag_value`).

- **Aliasing the Column:** The syntax requires you to assign an alias to the matched column using `AS` alias. Looking at the rest of the query prompt, the final clause specifies `ON COLUMN cc`; Therefore, the alias defined in the `MATCH COLUMNS` statement must be `cc` so that it aligns correctly with the target column placeholder.

Why the other options are incorrect:

- `MATCH COLUMNS hasTag('pii') AS credit_card`: While `hasTag('pii')` checks if the tag key exists, it fails to filter specifically for the value `'credit_card'`, and it assigns the wrong alias (`credit_card` instead of `cc`).

- MATCH COLUMNS hasTagValue('pii') AS credit_card: The hasTagValue function requires both the key and the value as arguments, and the alias is wrong.

- MATCH COLUMNS hasTagValue('pii', 'cc') AS credit_card: This incorrectly attempts to find a tag value of 'cc' and sets the alias to credit_card instead of cc.

Question 43

Which of the following locations completely hosts the customer data?

- A. Databricks-managed cluster
- B. Control plane
- C. Databricks account
- D. Customer's cloud account

Correct answer: D

- D. Customer's cloud account

Explanation

According to the Databricks Lakehouse architecture, the storage account hosting the customer data is provisioned in the data plane in the Databricks customer's cloud account.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 44

A data engineer is joining a very large PySpark dataframe with a small dataframe, as follows:

```
largeDF.join(smallerDF, ["key"], "inner")
```

This join operation is slow due to data shuffling. They want to optimize it by allowing the smaller dataframe to be sent to all executor nodes in the cluster.

Which of the following functions can be used to mark a dataframe as small enough to fit in memory on all executors?

- A. `pyspark.sql.functions.shuffle`
- B. `pyspark.sql.functions.explode`
- C. `pyspark.sql.functions.distribute`
- D. `pyspark.sql.functions.broadcast`

Correct answer: D

D. `pyspark.sql.functions.broadcast`

Explanation

`Pyspark.sql.functions.broadcast` function marks a DataFrame as small enough for use in broadcast joins.

In Apache Spark, joining a large DataFrame with a small DataFrame typically triggers a **Shuffle Hash Join** or **Sort Merge Join**. These operations require moving data across the network (shuffling) so that rows with matching keys end up on the same executor node, which is highly resource-intensive and slows down performance.

By wrapping the smaller DataFrame in the `broadcast()` function, you explicitly instruct Spark to use a **Broadcast Hash Join** (also known as a map-side join). Instead of shuffling the large dataset, Spark copies the entire small dataset to every executor node in the cluster. The large dataset can then be read sequentially, and the join happens locally in memory on each executor, completely eliminating the expensive network shuffle phase.

Your optimized code would look like this:

```
from pyspark.sql.functions import broadcast

largeDF.join(broadcast(smallerDF), ["key"], "inner")
```

Why the Other Options Are Incorrect

- **distribute**: This is not a PySpark function used for joins. In Spark SQL, `DISTRIBUTE BY` is used to control how data is partitioned across executors based on a key, but it does not broadcast data.

- **explode**: This function takes a column containing an array or a map and outputs a new row for each element in that array or map. It is used for flattening data, not for optimizing joins.

- **shuffle**: This function randomly permutes the elements of an array column. It does not control cluster-wide data movement or join strategies.

Question 45

A data engineer has a Job with multiple tasks that takes more than 2 hours to complete. In the last run, the final task unexpectedly failed.

Which of the following actions can the data engineer perform to complete this Job Run while minimizing the execution time?

- A. They can keep the failed Run, and simply start a new Run for the Job
- B. They need to delete the failed Run, and start a new Run for the Job
- C. They can run the Job in Production mode which automatically retries execution in case of errors
- D. They can repair this Job Run so only the failed tasks will be re-executed

Correct answer: D

- D. They can repair this Job Run so only the failed tasks will be re-executed

Explanation

You can repair failed multi-task jobs by running only the subset of unsuccessful tasks and any dependent tasks. Because successful tasks are not re-run, this feature reduces the time and resources required to recover from unsuccessful job runs.

* Please note that Databricks Jobs has been recently renamed to Lakeflow Jobs, however, the current exam version may still refer to it as Databricks Jobs.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 46

A data pipeline team notices that several scheduled job runs are delayed during peak hours and start later than their scheduled time.

What is the most likely explanation for this behavior?

- A. The task dependencies are causing circular execution delays
- B. The job is configured with a low `timeout_seconds` value causing delayed retries
- C. The cluster is using spot instances that are frequently terminated
- D. The workflow is configured with queueing enabled and has reached its `max_concurrent_runs` limit

Correct answer: D

D. The workflow is configured with queueing enabled and has reached its `max_concurrent_runs` limit

Explanation

By default (or when configured manually), Databricks features a **Job Queueing** mechanism. When a job triggers frequently or experiences delays during "peak hours" and attempts to start a new run while previous runs are still active, Databricks checks the `max_concurrent_runs` parameter. If that concurrency threshold has already been reached, Databricks will place the subsequent runs into a QUEUED state for up to 48 hours rather than immediately failing or skipping them. As soon as a running slot clears up, the job is dequeued and executed, which perfectly explains why they **start later than their scheduled time**.

Why the other options are incorrect:

- **The job is configured with a low `timeout_seconds` value causing delayed retries**

- A low `timeout_seconds` value tells Databricks to actively terminate/cancel a job if it runs longer than the threshold. It causes an outright failure status (TIMEDOUT), not a delayed startup. While retries can happen if a retry policy is set, a low timeout would kill the job faster, not gracefully push back the scheduled start time of a pending job run.

- **The cluster is using spot instances that are frequently terminated**

- If a cluster loses a spot instance mid-run, Databricks has to acquire a new instance, causing the already running tasks to experience internal retries and long execution durations. This results in the job taking longer to finish, but it does not cause the job status to sit sequentially delayed at the initial startup gate.

- **The task dependencies are causing circular execution delays**

- Databricks Workflows strictly enforce Directed Acyclic Graph (DAG) structures. You are barred from

creating a circular dependency loop (e.g., Task A depends on Task B, which depends on Task A) in the UI or API config layout. If a pipeline cannot be structured cleanly due to a loop error, the workspace throws a configuration validation error preventing deployment altogether.

Question 47

A data engineering team is discussing the optimal data layout strategy on a growing managed Delta table in Unity Catalog. They are considering partitioning, Z-ordering, and Liquid Clustering to improve query performance.

Which scenario best indicates that Automatic Liquid Clustering is the recommended choice?

- A. The table is heavily filtered by a consistent, small set of date ranges.
- B. The team has identified stable clustering keys for the table.
- C. The table experiences diverse, frequently changing query filters across multiple columns, with unpredictable access patterns.
- D. None of the listed options are correct. Automatic Liquid Clustering can not be applied on managed tables.

Correct answer: C

C. The table experiences diverse, frequently changing query filters across multiple columns, with unpredictable access patterns.

Explanation

Automatic Liquid Clustering is designed to dynamically adapt to evolving and unpredictable query patterns by continuously reorganizing data based on recent query filters. This is especially beneficial when query predicates frequently change across multiple columns, making static strategies like partitioning or Z-ordering less effective.

Partitioning works best when filters are stable and predictable, often on date/time columns. Z-ordering optimizes clustering for known high-cardinality columns with consistent filtering. When query filters are varied and unpredictable, Automatic Liquid Clustering provides the agility to improve performance without manual tuning.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 48

Which TWO of the following are native capabilities of Delta table?

- A. Delta Lake prevents writes that do not conform to the table schema.
- B. Delta Lake can evolve the table schema to include new optional columns without having to manually recreate the table.
- C. Delta Lake enables efficient table data storage using JSON format.
- D. Delta Lake eliminates the need for any underlying storage system such as cloud object storage.
- E. Delta Lake automatically deletes old data versions after every write operation to save space.

Correct answers: A, B

- A. Delta Lake prevents writes that do not conform to the table schema.
- B. Delta Lake can evolve the table schema to include new optional columns without having to manually recreate the table.

Explanation

The two correct native capabilities of a Delta table are:

- **Delta Lake prevents writes that do not conform to the table schema.** (Known as Schema Enforcement or schema validation, this native safeguard rejects write operations that mismatch the target table's structure to guarantee data quality).
- **Delta Lake can evolve the table schema to include new optional columns without having to manually recreate the table.** (Known as Schema Evolution, this native capability allows users to seamlessly add new columns using options like mergeSchema without needing to rewrite the entire historical dataset).

Why other options are incorrect:

- **"Delta Lake enables efficient table data storage using JSON format."**
- Correction: Delta Lake uses **Apache Parquet** (a highly efficient columnar storage format) for actual data storage. Only its transaction log (`_delta_log`) is written using JSON.
- **"Delta Lake eliminates the need for any underlying storage system such as cloud object storage."**

- Correction: Delta Lake is an open-source storage layer that sits **on top of** an existing storage system (such as AWS S3, Azure ADLS, Google Cloud Storage, or HDFS). It does not replace them.

- **"Delta Lake automatically deletes old data versions after every write operation to save space."**

- Correction: It retains older data versions natively to support its "Time Travel" feature. Historical files are only removed when a user explicitly runs a VACUUM command.

Question 49

A data engineering team has the following Declarative Automation Bundle (DAB) configuration:

```
bundle:
  name: demo_bundle

variables:
  catalog_name:
    default: demo_catalog
```

The team wants to set the `catalog_name` variable per target to enable environment-specific behavior without modifying the business code.

Which configuration meets this requirement?

A.

```
targets:
  dev:
    variables:
      catalog_name: dev_catalog
  prod:
    variables:
      catalog_name: prod_catalog
```

B.

```
variables:
  catalog_name:
    default: demo_catalog
  targets:
    dev: dev_catalog
    prod: prod_catalog
```

C.

```
variables:
  catalog_name:
    default: demo_catalog
    dev: dev_catalog
    prod: prod_catalog
```

D.

```
variables:
  catalog_name:
    default: demo_catalog
    target: ${target}_catalog
```

Correct answer: A

Explanation

Custom variables are globally declared at the top level of the bundle configuration, but environment-specific values must be assigned by defining a variables mapping nested directly inside each specific deployment target under the `targets` block.

When you deploy using a specific target (e.g., `databricks bundle deploy -t dev`), the Databricks CLI automatically overrides the top-level default value of the variable with the value explicitly mapped inside

that active target block.

The other options are incorrect because they represent invalid syntax formats.

Question 50

A data engineer uses the following SQL query:

```
GRANT USE SCHEMA ON SCHEMA sales_db TO finance_team
```

Which of the following is the benefit of the **USE SCHEMA** privilege ?

- A. It's a prerequisite to perform any action on the database
- B. Gives the ability to view database objects and their metadata
- C. Gives read access on the database
- D. Gives full permissions on the entire database

Correct answer: A

- A. It's a prerequisite to perform any action on the database

Explanation

The `USE SCHEMA` privilege (formerly known as `USAGE`) does not give any direct permissions. However, it serves as a prerequisite for performing any actions on the objects within the schema. Additionally, it is necessary to grant the `USE CATALOG` privilege on the parent catalog to enable access in Unity Catalog.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 51

A data engineer needs to run a Lakeflow job that aggregates sales data once every night at 2 AM. The job does not require real-time processing and should execute only at a fixed time each day.

What trigger configuration should the data engineer use for this job?

- A. Table update trigger
- B. Scheduled trigger
- C. File arrival trigger
- D. Continuous trigger

Correct answer: B

B. Scheduled trigger

Explanation

In Lakeflow Jobs, the Scheduled trigger is designed specifically for batch processing tasks that need to run at predetermined, fixed intervals or times (such as once a night at 2 AM). It typically utilizes cron expressions or standard time pickers to orchestrate the execution without needing any external data event to prompt it.

Why the other options are incorrect:

- **Continuous trigger:** This is used for real-time or streaming pipelines where the job runs constantly to process data as soon as it arrives.

- **File arrival / Table update triggers:** These are event-driven triggers. They execute a job immediately after a specific event occurs (e.g., a new file lands in cloud storage or an upstream table finishes updating), rather than waiting for a specific time on the clock.

Question 52

A data engineering team is migrating a high-traffic external table to a managed table. They require a conversion process with minimal disruption to avoid impacting ongoing analytics workloads and business operations. To meet this requirement, they must decide between using `ALTER TABLE ... SET MANAGED` and a `DEEP CLONE` approach.

Which advantage does `SET MANAGED` provide over `DEEP CLONE` for this conversion?

Choose 2 answers

- A. `SET MANAGED` minimizes reader and writer downtime.
- B. `SET MANAGED` handles concurrent writes during conversion.
- C. `SET MANAGED` leaves data in place and just references the existing table location.
- D. `SET MANAGED` migrates only the Delta transaction logs to reduce data copy times.
- E. `SET MANAGED` schedules the conversion process to run during off-peak periods using predictive optimization.

Correct answers: A, B

- A. `SET MANAGED` minimizes reader and writer downtime.
- B. `SET MANAGED` handles concurrent writes during conversion.

Explanation

The correct two advantages that `SET MANAGED` provides over a `DEEP CLONE` approach are:

- **Minimizes reader and writer downtime:** The `SET MANAGED` command minimizes or eliminates downtime compared to alternative approaches like `DEEP CLONE`.
- **Handles concurrent writes during conversion:** The conversion process uses a two-step approach where table data and Delta transaction logs are initially copied while readers and writers continue operating normally with no impact to ongoing operations. Commits made to the external location during this period are safely caught up and moved to the managed location."

Why other options are incorrect:

- **"Leaves data in place...":** This describes a Shallow Clone or simply changing table properties without moving files. `SET MANAGED` physically copies data from the external storage location to the default Unity Catalog managed storage location.

- **"Migrates only the Delta transaction logs..."**: SET MANAGED copies both the underlying data files and the transaction logs to fully migrate the table into a managed state.

- **"Schedules the conversion process... using predictive optimization"**: Predictive optimization manages maintenance operations (like OPTIMIZE and VACUUM) after a table is managed, but it does not automatically predictively schedule the initial SET MANAGED migration process.

Question 53

A data engineer is validating incoming transaction data in a Databricks SQL pipeline. Some records contain unexpected symbols in numeric fields. The engineer runs the following query:

```
SELECT COALESCE(CAST("100$" AS INT), 0)
```

Which of the following describes the result of this query?

- A. 100
- B. An error is thrown
- C. 0
- D. NULL

Correct answer: B

B. An error is thrown

Explanation

COALESCE is a built-in SQL function that evaluates a list of arguments in order and **returns the very first non-NULL value** it finds, in this case, 0.

However, because you use the CAST function on a string that contains characters that are not part of the target type (such as the trailing \$ symbol in '100\$'), the call fails immediately and throws a runtime exception: **[CAST_INVALID_INPUT]**. So, the outer COALESCE() function is never successfully evaluated.

If the data engineer wants to tolerate malformed inputs and fallback to **0** instead of crashing the pipeline, they should use TRY_CAST() instead:

```
SELECT COALESCE(TRY_CAST("100$" AS INT), 0)
```

Question 54

A data engineer has an existing Databricks job and wants to manage it using Declarative Automation Bundles (formerly Databricks Asset Bundles). They want to:

Use the Databricks CLI to get the YAML definition of the job and download its referenced artifacts.

Automatically links the generated resource with the existing one in the workspace, ensuring that any updates to the bundle's YAML definition will modify the linked job rather than creating a new one.

Which of the following commands allows the data engineer to achieve this?

- A. `databricks bundle clone job --existing-job-id <job_id> --link`
- B. `databricks bundle download job --existing-job-id <job_id> --bind`
- C. `databricks bundle get job --existing-job-id <job_id> --link`
- D. `databricks bundle generate job --existing-job-id <job_id> --bind`

Correct answer: D

- D. `databricks bundle generate job --existing-job-id <job_id> --bind`

Explanation

The correct command is `databricks bundle generate`, because it allows the data engineer to generate bundle configuration for a resource that already exists in your Databricks workspace. This process generates a YAML definition of a job, pipeline, app, or dashboard and automatically downloads any artifacts it references, such as notebooks.

Flags:

- `--existing-job-id` int: Job ID of the job to generate config for
- `--bind`: Automatically bind the generated resource with the existing one in the workspace.

The other options are incorrect because these commands do not exist in the Databricks CLI syntax.

Question 55

In Delta Lake tables, which of the following is the primary format for the data files?

- A. JSON
- B. Delta
- C. Parquet
- D. Both, Parquet and JSON

Correct answer: C

C. Parquet

Explanation

Delta Lake builds upon standard data formats. Delta lake table gets stored on the storage in one or more data files in Parquet format, along with transaction logs in JSON format.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)